

Reference Notes

Week 1: While Loops

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

When should we choose one type of loop over another? You've mastered `for` loops, but what if you don't know exactly how many times to repeat? What if you want to keep going until something specific happens? This week, we discover a new kind of loop that's perfect for these situations!

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and using `for` loops (Module 1, Week 9)
- Using `range()` function (Module 1, Week 9)
- Writing conditional statements with `if` (Module 1, Week 7)
- Understanding comparison operators like `<`, `>`, `==` (Module 1, Week 7)

1.2 What You Already Know

You can use `for` loops to repeat code a specific number of times. You understand how `range()` creates sequences of numbers. You've written programs that make decisions using `if` statements and comparison operators. Now we'll combine these concepts in a powerful new way!

1.3 What You'll Be Able to Do

By the end of this week, you'll:

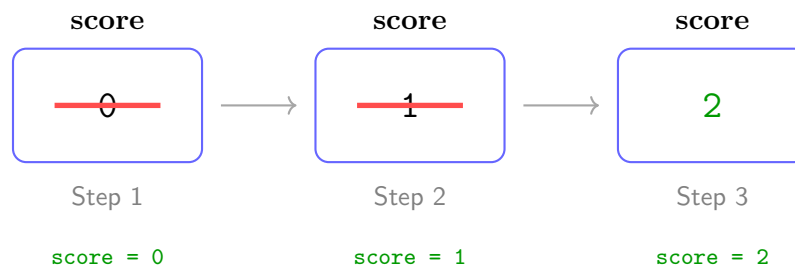
- Master `while` loops to repeat code based on conditions
- Distinguish when to use `while` vs `for` loops
- Implement proper exit conditions to avoid infinite loops
- Create input validation to ensure users enter correct data
- Build countdown timers and menu systems
- Debug `while` loops using trace tables and print statements

2 VIDEO 1: Memory Diagram Refresher

2.1 Quick Review: How Memory Diagrams Work

Before diving into while loops, let's refresh our memory diagram skills. You'll need these to understand how loop variables change!

```
1 # Quick review: How variables change in memory
2 score = 0           # Start with score containing 0
3 score = score + 1   # Add 1 to score, now contains the new value
4 score = score + 1   # Add 1 again, score now contains 2
5 print(score)        # Output: 2
```



Same variable, different values!

Remember: In Python, when we change a number, we create a new value. The variable contains the new value, and the old value disappears.

i Tip

Memory Diagram Quick Rules:

- Variables are containers that contain a single value
- When you reassign, old value no longer exists

3 VIDEO 2: Introduction to While Loops

3.1 Understanding the While Loop

A **while** loop repeats code as long as a condition is **True**. Unlike **for** loops that run a specific number of times, **while** loops continue until something changes. Think of it like waiting at a traffic light - you don't count seconds, you wait *while* the light is red!

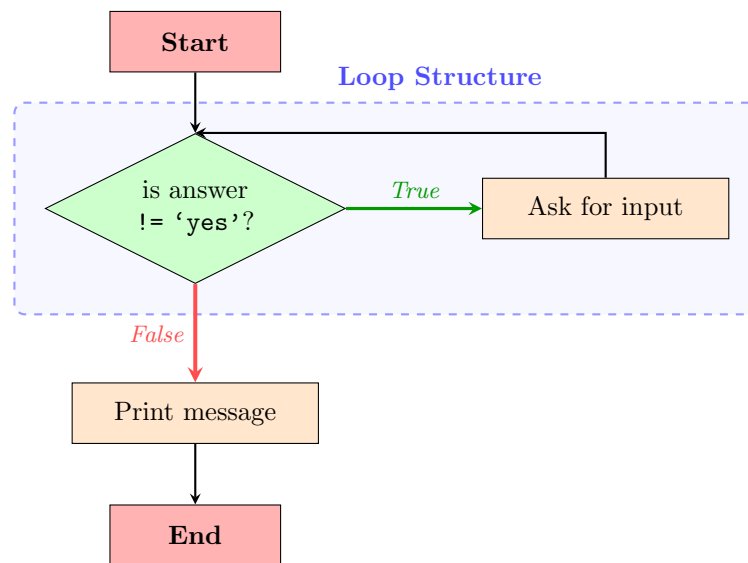
3.2 Your First While Loop

```
1 # Keep asking until user says yes
2 answer = ""
3 while answer != "yes":
4     answer = input("Do you want to continue? ")
5 print("Great! Let's continue!")
```

This loop keeps asking the question until the user types exactly "yes". We don't know how many times it will run - that depends on the user!

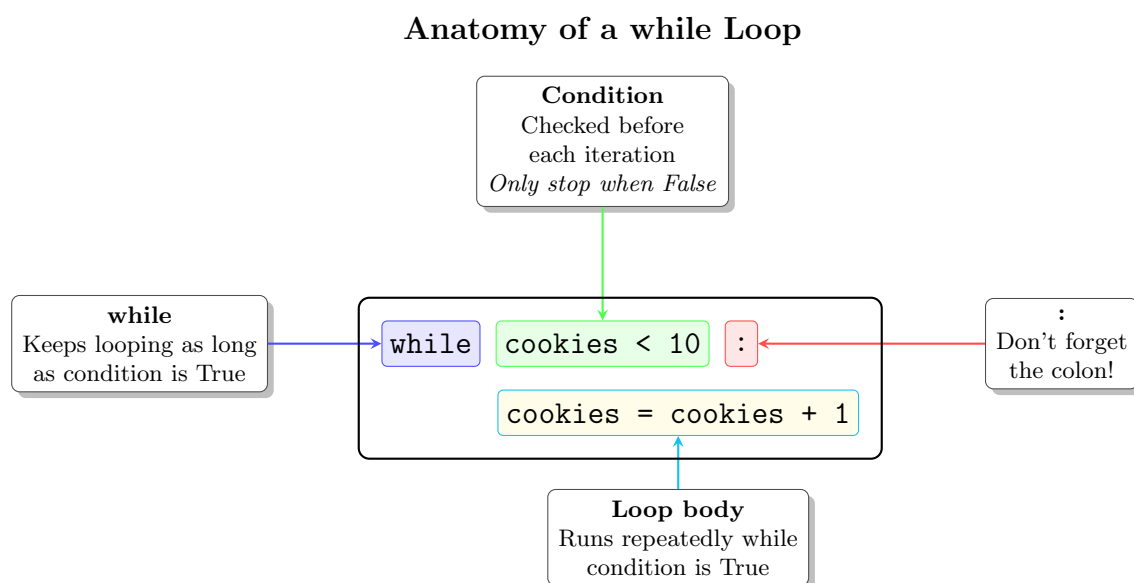
3.3 How It Works

This can be visualized with a flow diagram:



3.4 The Anatomy of a While Loop

The following diagram breaks down the components of a `while` loop:



The python syntax of the `while` loop consists of:

```

1 # Three essential parts:
2 count = 0           # 1. Initialize before loop
3 while count < 5:     # 2. Condition to check
4     print(f"Count: {count}")
5     count = count + 1 # 3. Update inside loop
6 print("Done!")
    
```

💡 Rule

The While Loop Template:

1. **Initialize** your variable(s) before the loop
2. **Check** the condition at the start of each iteration
3. **Update** the variable(s) inside the loop to eventually make the condition false

Forget any of these and you might create an infinite loop!

⚠️ Common Error

Common Mistake: Forgetting to initialize or update loop variables

If you forget to initialize:

```
1 while x < 10: # Error! x doesn't exist yet
2     print(x)
```

If you forget to update:

```
1 x = 0
2 while x < 10:
3     print(x) # Prints 0 forever - infinite loop!
```

👉 Quick Check

How many times will this loop run?

```
1 num = 10
2 while num > 0:
3     print(num)
4     num = num - 3
```

Trace through it: What values will num have?

4 VIDEO 3: While vs For Loops

4.1 Understanding the Difference

Both `while` and `for` loops repeat code, but they're designed for different situations. Choosing the right one makes your code clearer and less error-prone.

4.2 When You Know the Count: Use For

```
1 # Printing a pattern 5 times - we know it's 5
2 for i in range(5):
3     print("*" * (i + 1))
```

4.3 When You Have a Condition: Use While

```
1 # Keep playing until user wants to quit
2 play_again = "yes"
```

```

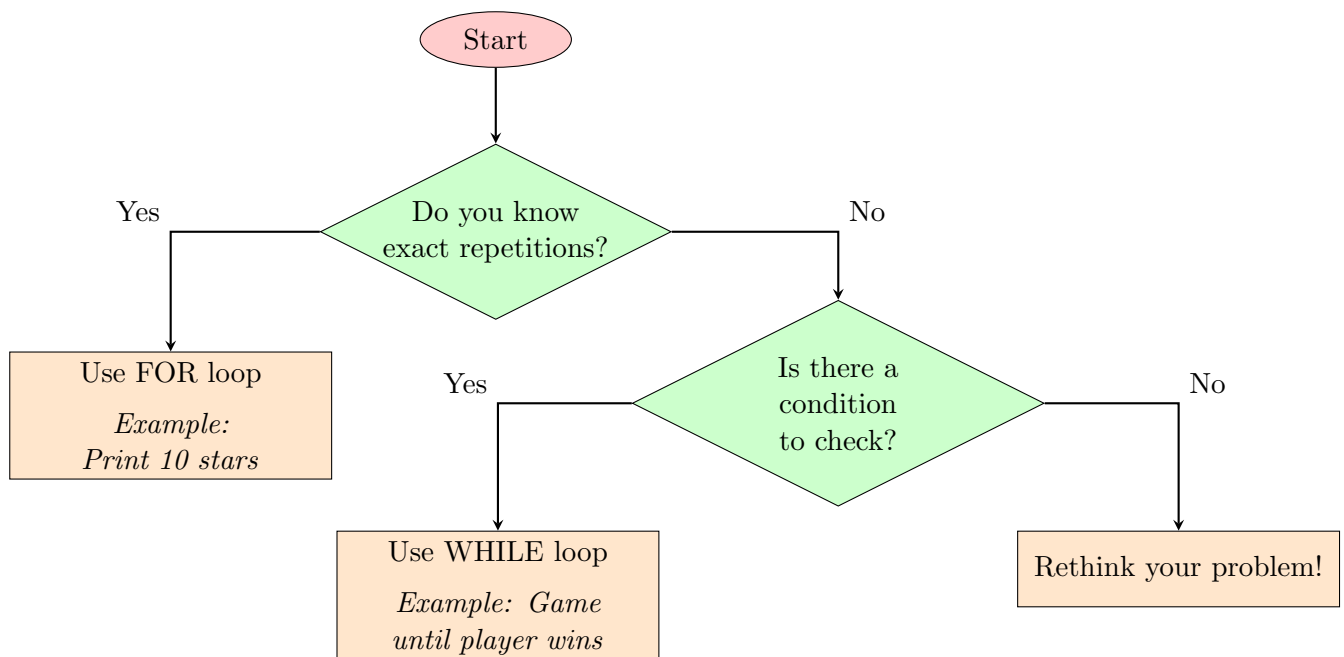
3 while play_again == "yes":
4     # Game code here
5     play_again = input("Play again? (yes/no): ")
6
7 # Continue until we reach a target
8 total = 0
9 while total < 100:
10     num = int(input("Enter a number: "))
11     total = total + num

```

4.4 Comparing Approaches

Scenario	Best Choice	Why
Print numbers 1-10	for loop	Known count (10 times)
Password validation	while loop	Unknown attempts needed
Process list items	for loop	Known items to process
Game main loop	while loop	Continues until quit condition
Countdown timer	for loop	Known iterations
Find first match	while loop	Stop when condition met

Visually it can be represented as:



i Tip

Quick Decision Guide:

- Counting? Iterating through items? → Use **for**
- Waiting for something? Checking conditions? → Use **while**
- Could use either? → **for** is usually safer (no infinite loops!)

Quick Check

Which loop would you choose for each task?

1. Print the times tables from 1 to 12
2. Keep asking for input until user types "quit"
3. Calculate the sum of the first 50 numbers
4. Roll dice until you get a 6

5 VIDEO 4: Loop Condition Patterns

5.1 Understanding Loop Conditions

The condition in a `while` loop determines when it continues and when it stops. Mastering different condition patterns lets you create sophisticated programs that respond intelligently to user input and changing data.

5.2 Pattern 1: Using a Special “Stop” Value

The following code asks the user to enter numbers and sums them until they enter -1 to stop:

```
1 # Keep going until a special "stop" value
2 total = 0
3 num = 0 # Initialize to enter loop
4 while num != -1: # -1 is our sentinel
5     num = int(input("Enter a number (-1 to stop): "))
6     if num != -1:
7         total = total + num
8 print(f"Sum: {total}")
```

5.3 Pattern 2: Flag Variables

The following code uses a boolean flag to control when the game loop should stop:

```
1 # Use a boolean flag to control the loop
2 game_running = True
3 score = 0
4 while game_running:
5     action = input("Action (play/quit): ")
6     if action == "quit":
7         game_running = False
8     elif action == "play":
9         score = score + 10
10 print(f"Final score: {score}")
```

5.4 Pattern 3: Validation Loops

The following code keeps asking the user for their age until they enter a valid number between 1 and 120:

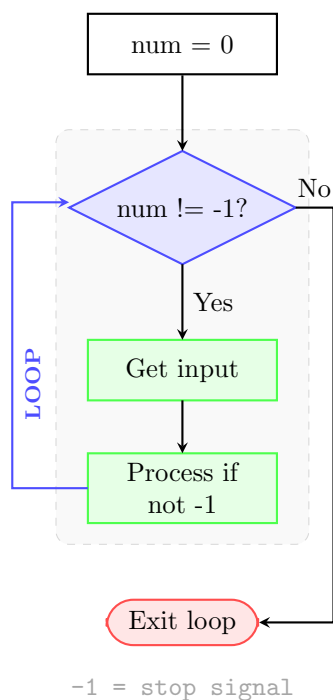
```

1 # Keep asking until valid input
2 age = 0
3 while age < 1 or age > 120:
4     age = int(input("Enter your age (1-120): "))
5     if age < 1 or age > 120:
6         print("Invalid age! Try again.")
7 print(f"Your age is {age}")

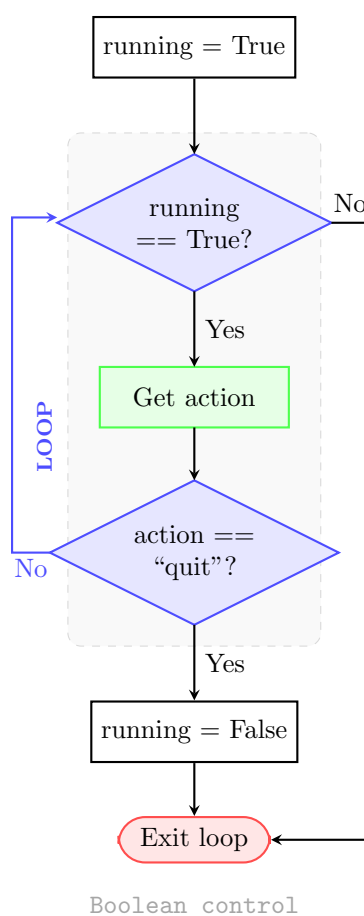
```

Three Ways to Control While Loops

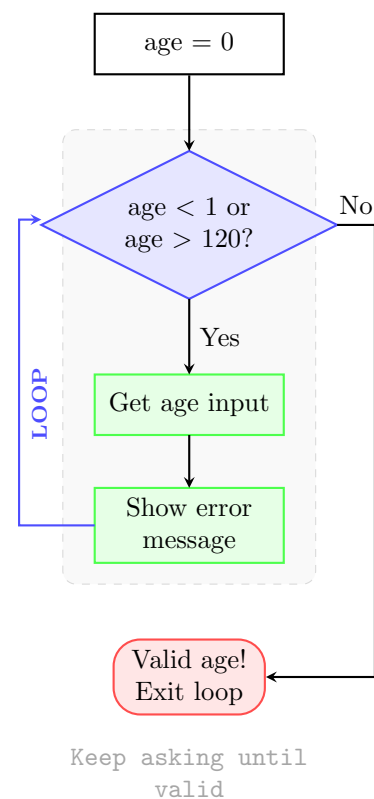
Pattern 1: Stop Value



Pattern 2: On/Off Switch



Pattern 3: Range Check



5.5 Pattern 4: Counter-Controlled While

Sometimes you use `while` even when you could use `for` - especially when the counter changes in complex ways:

```

1 # Double a number until it exceeds 1000
2 value = 1
3 attempts = 0
4 while value < 1000:
5     value = value * 2
6     attempts = attempts + 1
7     print(f"Attempt {attempts}: value = {value}")
8 print(f"Took {attempts} doublings")

```

⚠ Common Error

Common Mistake:

`while` and `for` loops are interchangeable, they're NOT! Each has its purpose:

- `for`: when you know the iteration count
- `while`: when you have a condition to check

Using the wrong one makes code harder to read and more error-prone.

5.6 Debugging While Loops

Infinite loops are the most common issue with `while` loops. If your loop never ends, check by adding debug print statements:

```
1 # Add debug prints to track what's happening
2 count = 0
3 target = 15
4 while count < target:
5     print(f"DEBUG: count={count}, target={target}")
6     count = count + 4 # Jumps by 4 each time
7 print(f"Final: count={count}")
```

👉 Quick Check

What's wrong with this loop? Will it ever stop?

```
1 x = 10
2 while x != 0:
3     x = x - 3
4     print(x)
```

Hint: Trace the values of `x`: 10, 7, 4, 1, -2...

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, you mastered `while` loops - a powerful tool for condition-based repetition! You can now create programs that adapt to user input, validate data, and continue running until specific conditions are met. You understand the crucial difference between `for` loops (for counting) and `while` loops (for conditions), and you know how to avoid the dreaded infinite loop.

6.2 Key Concepts We Covered

Concept	Key Points	Example Use
<code>while</code> loops	Repeat based on condition	Input validation
Loop initialization	Set up variables before loop	<code>count = 0</code>
Loop conditions	Control when to continue/stop	<code>while x < 10:</code>
Loop updates	Change variables to exit	<code>x = x + 1</code>
Sentinel values	Special values to stop	-1 to quit
Flag variables	Boolean loop control	<code>game_running</code>

6.3 Common Patterns to Remember

- **Input validation:** Keep asking until you get valid data
- **Menu loops:** Continue until user chooses to quit
- **Sentinel patterns:** Stop when a special value appears
- **Flag control:** Use boolean variables for clean loop control

★ Landmark Moment

You've unlocked a new dimension of programming! With `while` loops, your programs can now respond intelligently to changing conditions, validate user input, and create interactive experiences that adapt on the fly. You're no longer limited to fixed repetition counts - your loops can be as flexible as you need them to be!

7 Quick Reference

📖 Quick Reference

Week 1 Essential Patterns:

Basic While Loop:

- Pattern: `while condition:`
- Common use: Unknown iteration count
- Common mistake: Forgetting to update loop variable

Input Validation:

- Pattern: `while not valid: get_input()`
- Common use: Ensuring correct user input
- Debug tip: Print the condition variables

Sentinel Pattern:

- Pattern: `while value != sentinel:`
- Common use: Stop on special value
- Common mistake: Forgetting to check before processing

Flag Pattern:

- Pattern: `running = True; while running:`
- Common use: Clean loop control
- Debug tip: Print flag state changes

7.1 Reflection Questions

1. Think about a time this week when you accidentally created an infinite loop. What was missing - initialization, condition, or update? How did you fix it?

2. When deciding between `for` and `while`, what question do you ask yourself first?
3. The validation loop pattern keeps asking until it gets good input. Where could you use this in a real program you might write?
4. Compare debugging a `while` loop to debugging a `for` loop. Which is trickier? Why?
5. Look at your favorite video game or app. Can you identify where it might use `while` loops? (Hint: Think about menus, gameplay, and user input!)

Next week: We'll explore advanced loop control with `break` and `continue` statements, giving you even more precise control over your loops!